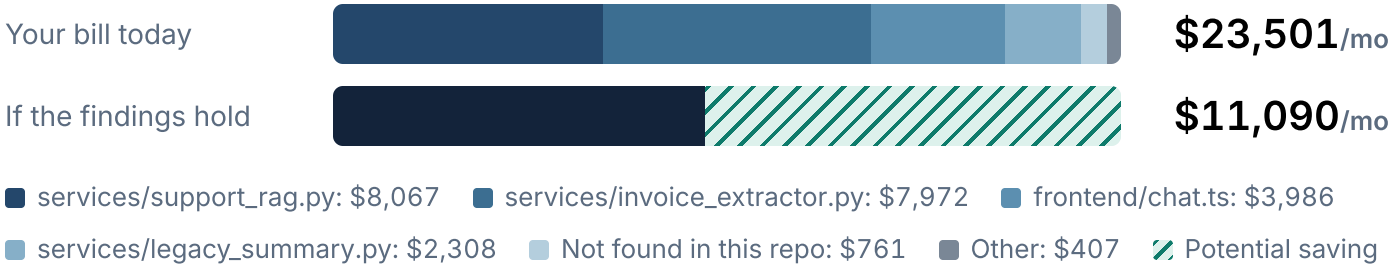
LLM cost report for demo_app

Found 7 LLM call sites in 7 files. Usage period: 2026-09-01 to 2026-09-30. Generated 2026-10-14 10:05 UTC.



$23,501
Measured monthly run-rate (30 days of data)

$5,629 – $12,412
Potential monthly saving, estimated, not yet tested

97%
Of spend traced to a line of code in this repo

What to change (9)

Sorted by severity, then by potential saving. Savings on the same line of code overlap, so the headline total counts only the largest one per call site.

Premium model `gpt-4o` on a simple task (extraction, structured_output)

save $3,747–$7,494/mo

F-001 `services/invoice_extractor.py:19` in `extract_invoice()`, `services/invoice_extractor.py:34` in `process_batch()`

high · slm · estimated · fine-tuned model candidate

`gpt-4o` costs ~17x more per token than `gpt-4o-mini`. Signals: extraction, structured_output.

> Test `gpt-4o-mini` (~17x cheaper per token) on a sample of real inputs with `trainlab experiment`. This is a narrow, repetitive task: a fine-tuned small model can usually match quality at a fraction of the cost.

Evidence

~1,902-token system prompt is not being cached

save $1,742–$3,484/mo

F-002 `services/support_rag.py:33` in `answer()`

high | cache | estimated

The static prompt is re-billed at full price on every call: dynamic content (e.g. retrieved documents) is placed before the static instructions, so the prompt prefix changes on every request and cannot be cached; no `cache_control` breakpoint is set, so the provider never caches it.

> Move static instructions to the start of the prompt and put per-request content after them. Add `cache_control: {"type": "ephemeral"}` on the static system block.

Evidence

`gpt-4` is near its deprecation date (2026-10-23)

$2,308/mo affected

F-003 `services/legacy_summary.py:10` in `summarize_notes()`

high | model | estimated

The provider has scheduled this model for retirement. Calls will fail after the shutdown date, and older models are usually priced above their successors.

> Migrate to the provider's current model; run `trainlab experiment` to compare quality first.

Evidence

Sequential calls in a loop in `process_batch`

save up to $1,993/mo

F-004 `services/invoice_extractor.py:34` in `process_batch()`

medium | batch | estimated

Requests are sent one at a time inside a loop. If results are not needed immediately, the provider Batch API processes them at a discount.

> If this workload tolerates up to 24h latency, submit it through the Batch API (~50% cheaper). Otherwise, run requests concurrently to cut wall time.

Evidence

Sequential calls in a loop in `summarize_notes`

save up to $1,154/mo

F-005 `services/legacy_summary.py:10` in `summarize_notes()`

medium · batch · estimated

Requests are sent one at a time inside a loop. If results are not needed immediately, the provider Batch API processes them at a discount.

> If this workload tolerates up to 24h latency, submit it through the Batch API (~50% cheaper). Otherwise, run requests concurrently to cut wall time.

Evidence

Premium model `gpt-4.1` on a simple task (classification)

save $140–$280/mo

F-006 `services/ticket_classifier.py:19` in `classify_ticket()`

medium · slm · estimated · fine-tuned model candidate

`gpt-4.1` costs ~13x more per token than `gpt-4o-mini`. Signals: classification.

> Test `gpt-4o-mini` (~13x cheaper per token) on a sample of real inputs with `trainlab experiment`. This is a narrow, repetitive task: a fine-tuned small model can usually match quality at a fraction of the cost.

Evidence

`claude-sonnet-4-5` is near its deprecation date (2026-11-30)

$8,067/mo affected

F-007 `services/support_rag.py:33` in `answer()`

medium · model · estimated

The provider has scheduled this model for retirement. Calls will fail after the shutdown date, and older models are usually priced above their successors.

> Migrate to the provider's current model; run `trainlab experiment` to compare quality first.

Evidence

Unbounded retries around an LLM call in `extract_invoice`

F-008 `services/invoice_extractor.py:19` in `extract_invoice()`

medium | reliability | from code only

The retry decorator has no stop condition. During a provider outage or a persistent error, this retries forever and every attempt is billed.

> Add a stop condition, e.g. `@retry(stop=stop_after_attempt(3), wait=wait_exponential())`.

Evidence

No output token limit

F-009 `services/invoice_extractor.py:19` in `extract_invoice()`, `services/invoice_extractor.py:34` in `process_batch()`

low | config | from code only

Without a cap, an unexpected long generation (or a reasoning model) is billed in full.

> Set `max_completion_tokens` (or `max_tokens`) to a safe upper bound for this task.

Candidates for a fine-tuned model

2 workload(s) are narrow and repetitive (extraction or classification), currently costing $8,275/month. A small model fine-tuned on your own examples typically handles these at a fraction of frontier-model cost, with predictable fixed pricing. First validate a cheaper off-the-shelf model with `trainlab experiment`; if quality falls short, a custom model is the next step.

- F-001: Premium model `gpt-4o` on a simple task (extraction, structured_output)
- F-006: Premium model `gpt-4.1` on a simple task (classification)

Spend by model

Model	Provider	Per month	Share	Requests	Tokens in / out	Cached input
`gpt-4o-2024-08-06`	openai	$11,959	51%	1.8M	2.7B / 531.5M	0%

Model	Provider	Per month	Share	Requests	Tokens in / out	Cached input
`claude-sonnet-4-5-20250929`	anthropic	$8,067	34%	—	1.5B / 239.0M	0%
`gpt-4-0613`	openai	$2,308	10%	59.2k	47.3M / 14.8M	0%
`text-embedding-3-large`	openai	$761	3%	1.5M	5.9B / 0	0%
`gpt-4.1`	openai	$302	1%	1.2M	141.7M / 2.4M	0%
`gpt-5-mini`	openai	$105	0%	299.6k	59.9M / 44.9M	0%

Spend by project

Project / workspace	Per month
`wrkspc_support`	$8,067
`proj_invoices`	$11,959
`proj_support`	$2,715
`proj_search`	$761

Every LLM call site

Location	Type	Model	Spend / month	Findings
`services/support_rag.py:33` answer()	chat	`claude-sonnet-4-5`	$8,067	F-002 F-007
`frontend/chat.ts:7`	chat	`gpt-4o`	$3,986 (shared)	
`services/invoice_extractor.py:19` extract_invoice()	chat	`gpt-4o`	$3,986 (shared)	F-001 F-008 F-009

Location	Type	Model	Spend / month	Findings
`services/invoice_extractor.py:34` process_batch()	chat	`gpt-4o`	$3,986 (shared)	F-001 F-004 F-009
`services/legacy_summary.py:10` summarize_notes()	chat	`gpt-4`	$2,308	F-003 F-005
`services/ticket_classifier.py:19` classify_ticket()	chat	`gpt-4.1`	$302	F-006
`services/well_tuned.py:10` short_reply()	chat	`gpt-5-mini`	$105	

Model names in configuration

Model names found outside call sites, e.g. in config files. They may be selected at runtime.

`config/models.yaml:2`	`claude-sonnet-4-5`
`config/models.yaml:3`	`text-embedding-3-large`

How to read this report

From code
Pattern found in source. No cost attached.

Estimated
Measured spend × price difference. Quality not yet tested.

Validated
Tested on your real inputs with `trainlab experiment`.

Realized
Confirmed in production billing after the change.

Spend is linked to code by model name: when several call sites use the same model, its spend is split evenly between them (marked "shared"). Imported traces that carry a source location (`code.file.path`, `code.line.number`) are attributed to that exact line (marked "traced").

Gaps in this analysis

$761/month of spend has no matching call site

F-010

info coverage estimated

These models appear in billing but not in the scanned code. They may be called from another repository, a notebook, a different service, or a runtime-configured model.

> Scan the other repositories that share this API key/project, or instrument at runtime.

Evidence

Generated locally by trainlab-optimize 0.1.3. No code or usage data left this machine. Prices: BerriAI/litellm model_prices_and_context_window.json (MIT), built 2026-10-14T08:36:59Z. Usage sources: anthropic_usage_file:anthropic_usage.json, csv:billing_export.csv, openai_usage_file:openai_usage_completions.json, openai_usage_file:openai_usage_embeddings.json.
Overlap: csv:billing_export.csv overlaps anthropic_usage_file:anthropic_usage.json, openai_usage_file:openai_usage_completions.json, openai_usage_file:openai_usage_embeddings.json for the same models and days; counted once.